

```
In [1]: # importing the necessary libraries

import pandas as pd, numpy as np
import seaborn as sns, matplotlib.pyplot as plt
import datetime as dt

import warnings
warnings.filterwarnings('ignore')
```

```
In [2]: # reading the data

odi = pd.read_csv("Cricket.csv", encoding = "ISO-8859-1")
odi.head()
```

```
Out[2]:
```

	Player	Span	Mat	Inns	NO	Runs	HS	Ave	BF	SR
0	SR Tendulkar (INDIA)	1989-2012	463	452	41	18426	200*	44.83	21367	86.6
1	KC Sangakkara (Asia/ICC/SL)	2000-2015	404	380	41	14234	169	41.98	18048	78.8
2	RT Ponting (AUS/ICC)	1995-2012	375	365	39	13704	164	42.03	17046	80.3
3	ST Jayasuriya (Asia/SL)	1989-2011	445	433	18	13430	189	32.36	14725	91.2
4	DPMD Jayawardene (Asia/SL)	1998-2015	448	418	39	12650	144	33.37	16020	78.9

```
In [3]: # data inspections

odi.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 79 entries, 0 to 78
Data columns (total 13 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   Player      79 non-null    object
 1   Span        79 non-null    object
 2   Mat         79 non-null    int64
 3   Inns        79 non-null    int64
 4   NO          79 non-null    int64
 5   Runs        79 non-null    int64
 6   HS          79 non-null    object
 7   Ave         79 non-null    float64
 8   BF          79 non-null    int64
 9   SR          79 non-null    float64
10  100         79 non-null    int64
11  50          79 non-null    int64
12  0           79 non-null    int64
dtypes: float64(2), int64(8), object(3)
memory usage: 8.2+ KB
```

```
In [4]: odi.shape
```

Out[4]: (79, 13)

In [5]: `odi.isnull().sum()`

Out[5]:

Player	0
Span	0
Mat	0
Inns	0
NO	0
Runs	0
HS	0
Ave	0
BF	0
SR	0
100	0
50	0
0	0

dtype: int64

In [6]: `odi.describe().T`

Out[6]:

	count	mean	std	min	25%	50%	75%
Mat	79.0	245.075949	74.211716	128.00	188.000	232.00	281.500
Inns	79.0	230.544304	70.321022	127.00	177.000	217.00	261.500
NO	79.0	30.037975	14.421710	4.00	17.500	29.00	40.000
Runs	79.0	7618.139241	2551.873313	5080.00	5759.000	6798.00	8739.500
Ave	79.0	38.523291	5.919093	23.57	34.600	37.87	41.845
BF	79.0	9684.455696	3193.835825	5504.00	7393.500	9134.00	10976.000
SR	79.0	79.295316	9.925307	60.57	73.725	77.73	85.180
100	79.0	11.556962	8.092014	0.00	6.000	10.00	15.500
50	79.0	46.443038	16.351701	23.00	34.500	42.00	54.000
0	79.0	13.253165	5.925755	3.00	9.000	13.00	16.000

In [7]: `odi.nunique()`

Out[7]:

Player	79
Span	65
Mat	62
Inns	69
NO	42
Runs	79
HS	62
Ave	77
BF	79
SR	77
100	28
50	41
0	22

dtype: int64

```
In [8]: grouped_odi = odi[['Player', 'Ave', 'SR']]
grouped_odi.head()
```

```
Out [8]:
```

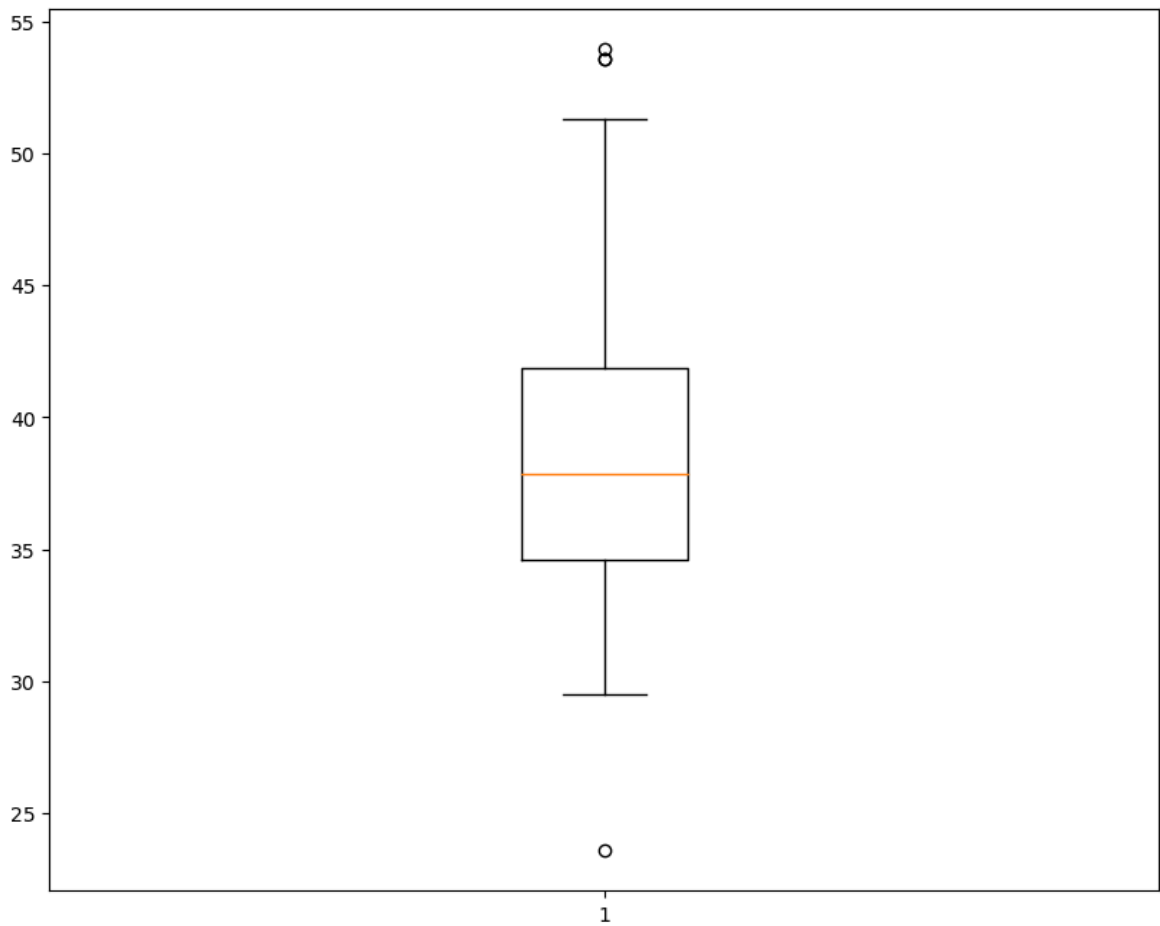
	Player	Ave	SR
0	SR Tendulkar (INDIA)	44.83	86.23
1	KC Sangakkara (Asia/ICC/SL)	41.98	78.86
2	RT Ponting (AUS/ICC)	42.03	80.39
3	ST Jayasuriya (Asia/SL)	32.36	91.20
4	DPMD Jayawardene (Asia/SL)	33.37	78.96

```
In [9]: grouped_odi.describe()
```

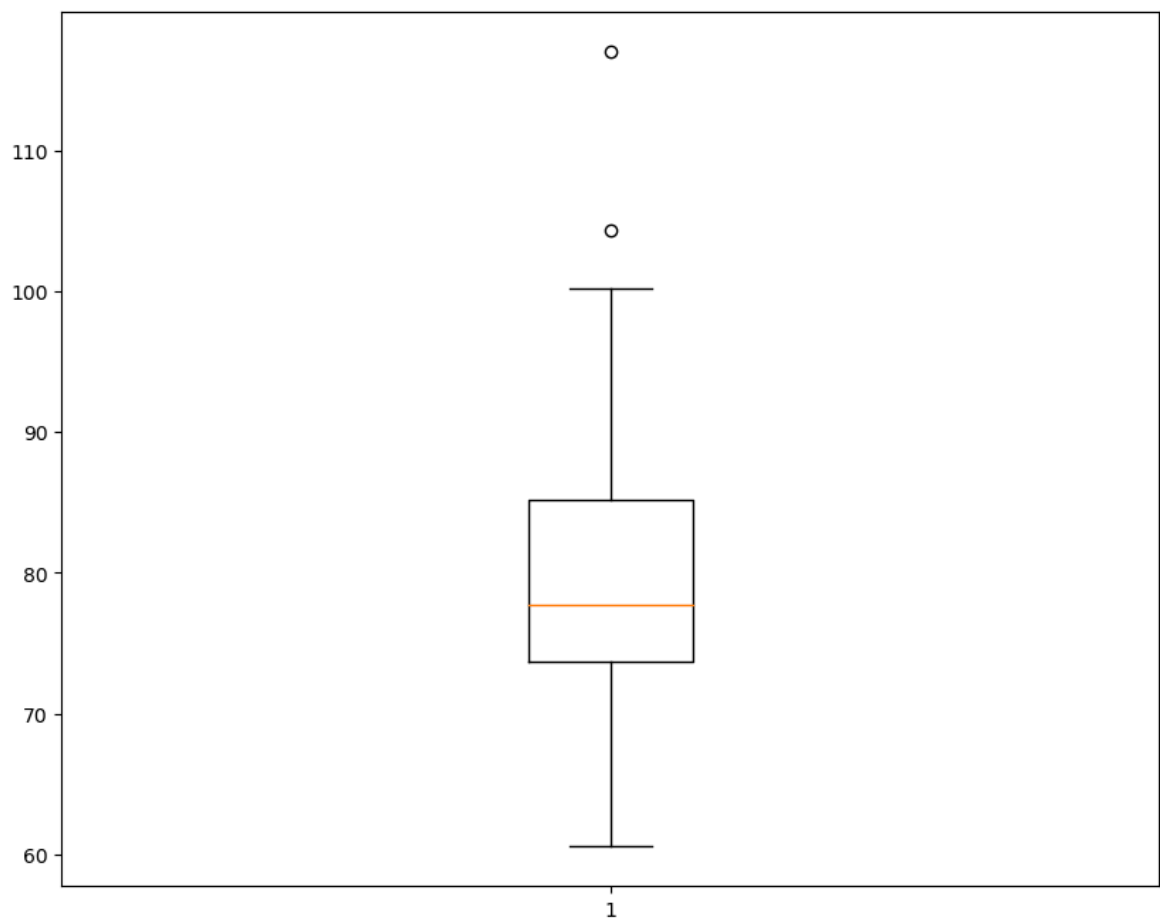
```
Out [9]:
```

	Ave	SR
count	79.000000	79.000000
mean	38.523291	79.295316
std	5.919093	9.925307
min	23.570000	60.570000
25%	34.600000	73.725000
50%	37.870000	77.730000
75%	41.845000	85.180000
max	53.940000	117.000000

```
In [10]: # checking for outliers
plt.figure(figsize = [10,8])
plt.boxplot(grouped_odi['Ave'])
plt.show()
```



```
In [11]: plt.figure(figsize = [10,8])  
plt.boxplot(grouped_odi['SR'])  
plt.show()
```



In [12]: *# Creating the clusters*

```
import sklearn
from sklearn.preprocessing import StandardScaler
```

In [13]: *# Need to scale the data before performing any analysis.*

```
df = grouped_odi[['Ave', 'SR']]
scaler = StandardScaler()
odi_scaled = scaler.fit_transform(df)
odi_scaled[:10]
```

Out[13]: array([[1.07229395, 0.70315153],
[0.58772461, -0.0441395],
[0.59622582, 0.1109969],
[-1.04790945, 1.20709133],
[-0.87618487, -0.03399986],
[0.16946476, -0.51259058],
[0.99238251, -0.64947564],
[0.42450125, -0.56734461],
[0.108256 , -0.8167796],
[0.33268811, 0.02176813]])

In [14]: `odi_scaled.shape`

Out[14]: (79, 2)

In [15]: `odi_scaled = pd.DataFrame(odi_scaled)`
`odi_scaled.columns = ['Average', 'Strike Rate']`
`odi_scaled.head()`

Out[15]:

	Average	Strike Rate
0	1.072294	0.703152
1	0.587725	-0.044139
2	0.596226	0.110997
3	-1.047909	1.207091
4	-0.876185	-0.034000

In [16]: *# Build the KMeans Clusters , making an assumption that K = 4*
`from sklearn.cluster import KMeans`
`kmeans = KMeans(n_clusters = 4, max_iter=50, random_state = 42)`
`kmeans.fit(odi_scaled)`

File "/opt/conda/envs/anaconda-panel-2023.05-py310/lib/python3.11/site-packages/joblib/externals/loky/backend/context.py", line 217, in _count_physical_cores
 raise ValueError(

Out[16]:

KMeans

KMeans(max_iter=50, n_clusters=4, random_state=42)

In [17]: `kmeans.labels_`

```
Out[17]: array([2, 0, 0, 1, 3, 0, 0, 0, 0, 0, 1, 0, 1, 2, 3, 2, 3, 1, 0, 0, 1, 0,
                3, 0, 1, 2, 1, 1, 3, 0, 3, 3, 0, 3, 2, 3, 3, 0, 2, 0, 3, 3, 2, 3,
                3, 3, 0, 3, 3, 0, 1, 0, 3, 3, 3, 3, 0, 1, 3, 1, 3, 1, 0, 2, 2, 0,
                0, 3, 0, 3, 1, 3, 3, 3, 0, 0, 3, 1, 3], dtype=int32)
```

```
In [18]: len(kmeans.labels_)
```

```
Out[18]: 79
```

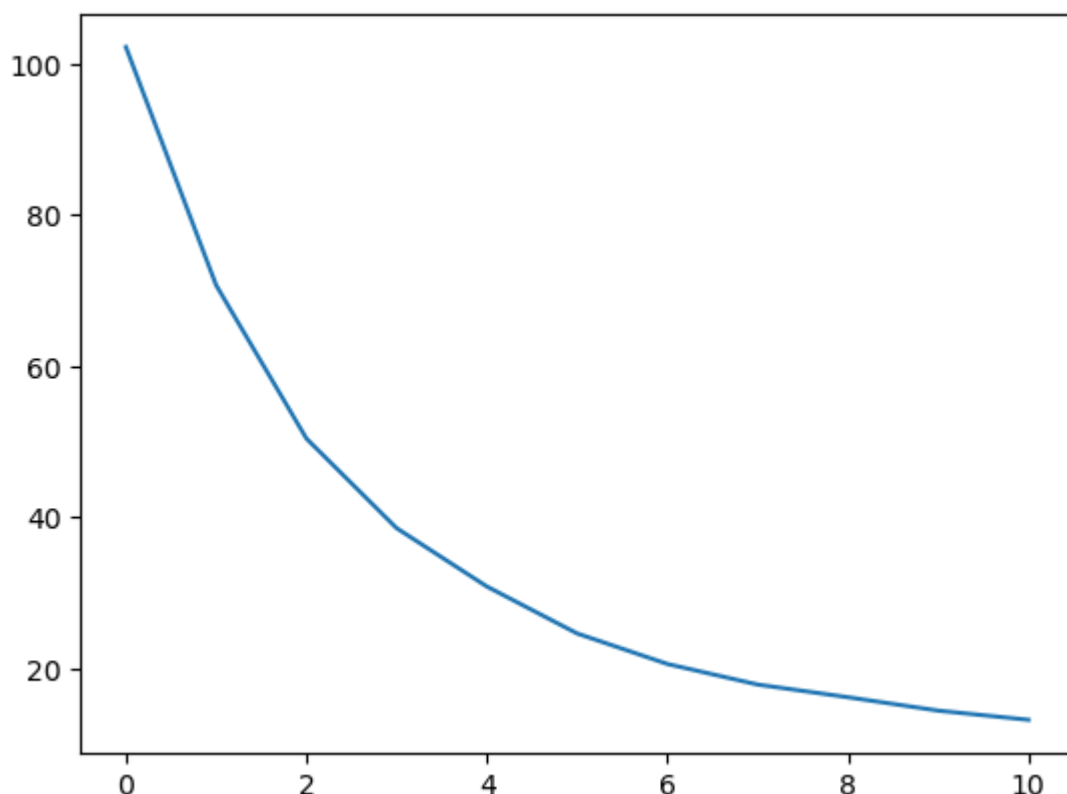
```
In [19]: # Elbow Method

ssd = []
range_n_clusters = [2,3,4,5,6,7,8,9,10,11,12]
for num_clusters in range_n_clusters:
    kmeans = KMeans(n_clusters = num_clusters, max_iter=50, random_state
                    kmeans.fit(odi_scaled)
                    ssd.append(kmeans.inertia_)
ssd
```

```
Out[19]: [102.22538186893445,
          70.70129909525369,
          50.413618426380395,
          38.54501170293659,
          30.822487862441513,
          24.610350173643283,
          20.573194043872093,
          17.851971469250092,
          16.175395271963332,
          14.4012892026859,
          13.18798503509532]
```

```
In [20]: plt.plot(ssd)
```

```
Out[20]: [<matplotlib.lines.Line2D at 0x7d4cf0fb3210>]
```



```
In [21]: from sklearn.metrics import silhouette_score
range_n_clusters = [2,3,4,5,6,7,8,9,10,11,12]
for num_clusters in range_n_clusters:
    kmeans = KMeans(n_clusters = num_clusters, max_iter=50, random_state
    kmeans.fit(odi_scaled)
    cluster_labels = kmeans.labels_
    silhoutte_avg = silhouette_score(odi_scaled,cluster_labels)
    print("For n_clusters = {0}, the silhouette score is {1}".format(num_
```

```
For n_clusters = 2, the silhouette score is 0.37632823237612395
For n_clusters = 3, the silhouette score is 0.37185804193936567
For n_clusters = 4, the silhouette score is 0.3885383317970877
For n_clusters = 5, the silhouette score is 0.36941231708738775
For n_clusters = 6, the silhouette score is 0.37909850189639954
For n_clusters = 7, the silhouette score is 0.38301845780271
For n_clusters = 8, the silhouette score is 0.3891868928159903
For n_clusters = 9, the silhouette score is 0.3919972629668421
For n_clusters = 10, the silhouette score is 0.37285754666257115
For n_clusters = 11, the silhouette score is 0.3345871786684004
For n_clusters = 12, the silhouette score is 0.3475030960279256
```

```
In [22]: # Now that we understand that we need 3 clusters by looking at the avg sc

kmeans = KMeans(n_clusters = 3, max_iter=50, random_state = 42)
kmeans.fit(odi_scaled)
```

```
Out [22]: KMeans
KMeans(max_iter=50, n_clusters=3, random_state=42)
```

```
In [23]: grouped_odi['Cluster_id'] = kmeans.labels_
grouped_odi
```

```
Out [23]:
```

	Player	Ave	SR	Cluster_id
0	SR Tendulkar (INDIA)	44.83	86.23	0
1	KC Sangakkara (Asia/ICC/SL)	41.98	78.86	0
2	RT Ponting (AUS/ICC)	42.03	80.39	0
3	ST Jayasuriya (Asia/SL)	32.36	91.20	2
4	DPMD Jayawardene (Asia/SL)	33.37	78.96	1
...	...	...	...	...
74	CG Greenidge (WI)	45.03	64.92	0
75	Misbah-ul-Haq (PAK)	43.40	73.75	0
76	PD Collingwood (ENG)	35.36	76.98	1
77	A Symonds (AUS)	39.75	92.44	2
78	Abdul Razzaq (Asia/PAK)	29.70	81.25	1

79 rows × 4 columns

```
In [24]: grouped_odi.Cluster_id.nunique()
```

```
In [25]: grouped_odi[['Player', 'Cluster_id']]
```

Out [25] :	Player	Cluster_id
0	SR Tendulkar (INDIA)	0
1	KC Sangakkara (Asia/ICC/SL)	0
2	RT Ponting (AUS/ICC)	0
3	ST Jayasuriya (Asia/SL)	2
4	DPMD Jayawardene (Asia/SL)	1
...	...	...
74	CG Greenidge (WI)	0
75	Misbah-ul-Haq (PAK)	0
76	PD Collingwood (ENG)	1
77	A Symonds (AUS)	2
78	Abdul Razzaq (Asia/PAK)	1

79 rows x 2 columns

```
In [26]: import plotly.express as px
fig = px.scatter(grouped_odi, x = 'Ave', y = 'SR', trendline = 'ols', hover_data = ['ODI'],
                  title = "ODI Clusters", width = 1000, height = 700)
fig.show()
```



```
In [27]: grouped_odi[grouped_odi['Cluster_id'] == 0]
```

Out [27]:

	Player	Ave	SR	Cluster_id
0	SR Tendulkar (INDIA)	44.83	86.23	0
1	KC Sangakkara (Asia/ICC/SL)	41.98	78.86	0
2	RT Ponting (AUS/ICC)	42.03	80.39	0
6	JH Kallis (Afr/ICC/SA)	44.36	72.89	0
9	BC Lara (ICC/WI)	40.48	79.51	0
11	Mohammad Yousuf (Asia/PAK)	41.71	75.10	0
13	MS Dhoni (Asia/INDIA)	51.32	88.69	0
15	AB de Villiers (Afr/SA)	53.55	100.25	0
25	V Kohli (INDIA)	53.94	90.99	0
29	MJ Clarke (AUS)	44.58	78.98	0
34	HM Amla (SA)	50.25	89.05	0
38	MG Bevan (AUS)	53.58	74.16	0
42	IVA Richards (WI)	47.00	90.20	0
46	LRPL Taylor (NZ)	44.12	82.03	0
49	ML Hayden (AUS/ICC)	43.80	78.96	0
51	DM Jones (AUS)	44.61	72.56	0
56	RR Sarwan (WI)	42.67	75.74	0
63	MJ Guptill (NZ)	43.30	87.73	0
64	MEK Hussey (AUS)	48.15	87.16	0
65	RG Sharma (INDIA)	42.46	84.56	0
68	DR Martyn (AUS)	40.80	77.73	0
70	G Gambhir (INDIA)	39.68	85.25	0
74	CG Greenidge (WI)	45.03	64.92	0
75	Misbah-ul-Haq (PAK)	43.40	73.75	0

In [28]: `grouped_odi[grouped_odi['Cluster_id'] == 1]`

Out [28]:

	Player	Ave	SR	Cluster_id
4	DPMD Jayawardene (Asia/SL)	33.37	78.96	1
5	Inzamam-ul-Haq (Asia/PAK)	39.52	74.24	1
7	SC Ganguly (Asia/INDIA)	41.02	73.70	1
8	R Dravid (Asia/ICC/INDIA)	39.16	71.24	1
14	M Azharuddin (INDIA)	36.92	74.02	1
16	PA de Silva (SL)	34.90	81.13	1
18	Saeed Anwar (PAK)	39.21	80.67	1
19	S Chanderpaul (WI)	41.60	70.74	1
21	DL Haynes (WI)	41.37	63.09	1
22	MS Atapattu (SL)	37.57	67.72	1
23	ME Waugh (AUS)	39.35	76.90	1
26	HH Gibbs (SA)	36.13	83.26	1
28	SP Fleming (ICC/NZ)	32.40	71.49	1
30	SR Waugh (AUS)	32.90	75.91	1
31	A Ranatunga (SL)	35.84	77.90	1
32	Javed Miandad (PAK)	41.70	67.01	1
33	Younis Khan (PAK)	31.24	75.29	1
35	Saleem Malik (PAK)	32.88	76.41	1
36	NJ Astle (NZ)	34.92	72.64	1
37	GC Smith (Afr/SA)	37.98	80.81	1
39	G Kirsten (SA)	40.95	72.04	1
40	A Flower (ZIM)	35.34	74.59	1
41	Shoaib Malik (PAK)	35.23	81.72	1
43	GW Flower (ZIM)	33.52	67.58	1
44	Ijaz Ahmed (PAK)	32.33	80.30	1
45	AR Border (AUS)	30.62	71.42	1
47	RB Richardson (WI)	33.41	63.74	1
48	WU Tharanga (Asia/SL)	34.30	75.03	1
52	DC Boon (AUS)	37.04	65.13	1
53	JN Rhodes (SA)	35.11	80.90	1
54	Mohammad Hafeez (PAK)	33.01	75.58	1
55	Rameez Raja (PAK)	32.09	63.31	1
58	CL Hooper (WI)	35.34	76.63	1
60	Tamim Iqbal (BDESH)	34.38	78.31	1

	Player	Ave	SR	Cluster_id
62	WJ Cronje (SA)	38.64	76.47	1
66	IR Bell (ENG)	37.87	77.16	1
67	A Jadeja (INDIA)	37.47	69.80	1
69	BRM Taylor (ZIM)	34.82	74.47	1
71	ADR Campbell (ZIM)	30.50	66.18	1
72	MN Samuels (WI)	34.30	76.24	1
73	RS Mahanama (SL)	29.49	60.57	1
76	PD Collingwood (ENG)	35.36	76.98	1
78	Abdul Razzaq (Asia/PAK)	29.70	81.25	1

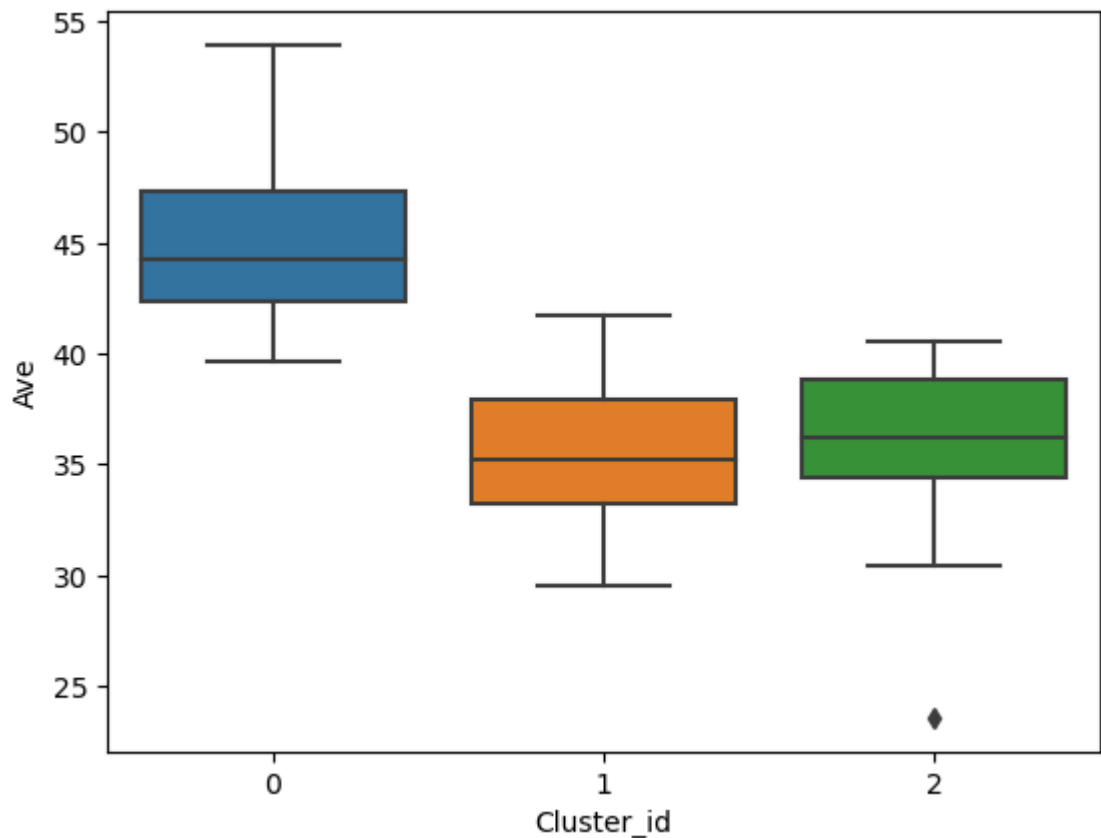
In [29]: `grouped_odi[grouped_odi['Cluster_id'] == 2]`

Out [29]:

	Player	Ave	SR	Cluster_id
3	ST Jayasuriya (Asia/SL)	32.36	91.20	2
10	TM Dilshan (SL)	39.27	86.23	2
12	AC Gilchrist (AUS/ICC)	35.89	96.94	2
17	CH Gayle (ICC/WI)	37.33	85.11	2
20	Yuvraj Singh (Asia/INDIA)	36.55	87.67	2
24	V Sehwag (Asia/ICC/INDIA)	35.05	104.33	2
27	Shahid Afridi (Asia/ICC/PAK)	23.57	117.00	2
50	BB McCullum (NZ)	30.41	96.37	2
57	EJG Morgan (ENG/IRE)	38.73	88.62	2
59	SR Watson (AUS)	40.54	90.44	2
61	SK Raina (INDIA)	35.46	93.76	2
77	A Symonds (AUS)	39.75	92.44	2

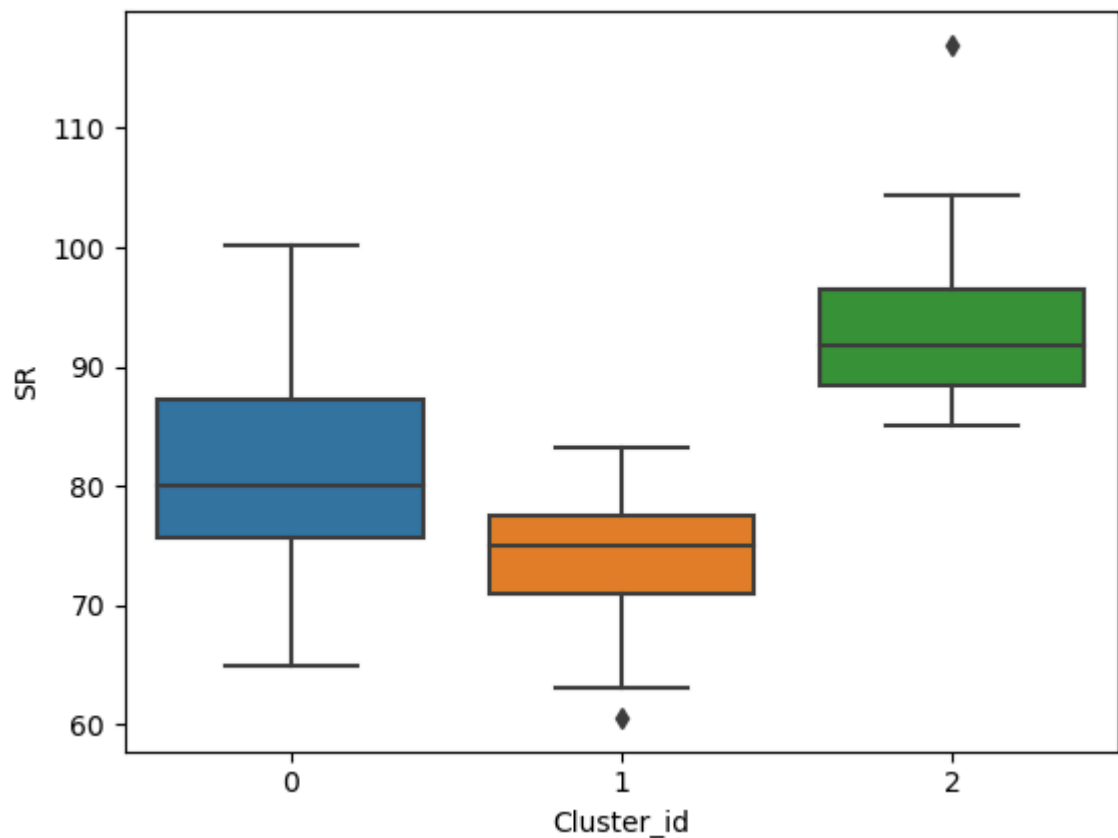
In [30]: `sns.boxplot(x= 'Cluster_id', y = 'Ave', data = grouped_odi)`

Out [30]: `<Axes: xlabel='Cluster_id', ylabel='Ave'>`



```
In [31]: sns.boxplot(x= 'Cluster_id', y = 'SR', data = grouped_odi)
```

```
Out[31]: <Axes: xlabel='Cluster_id', ylabel='SR'>
```



```
In [32]: from scipy.cluster.hierarchy import linkage, dendrogram, cut_tree
```

```
In [33]: odi_scaled.head()
```

Out [33]:

	Average	Strike Rate
0	1.072294	0.703152
1	0.587725	-0.044139
2	0.596226	0.110997
3	-1.047909	1.207091
4	-0.876185	-0.034000

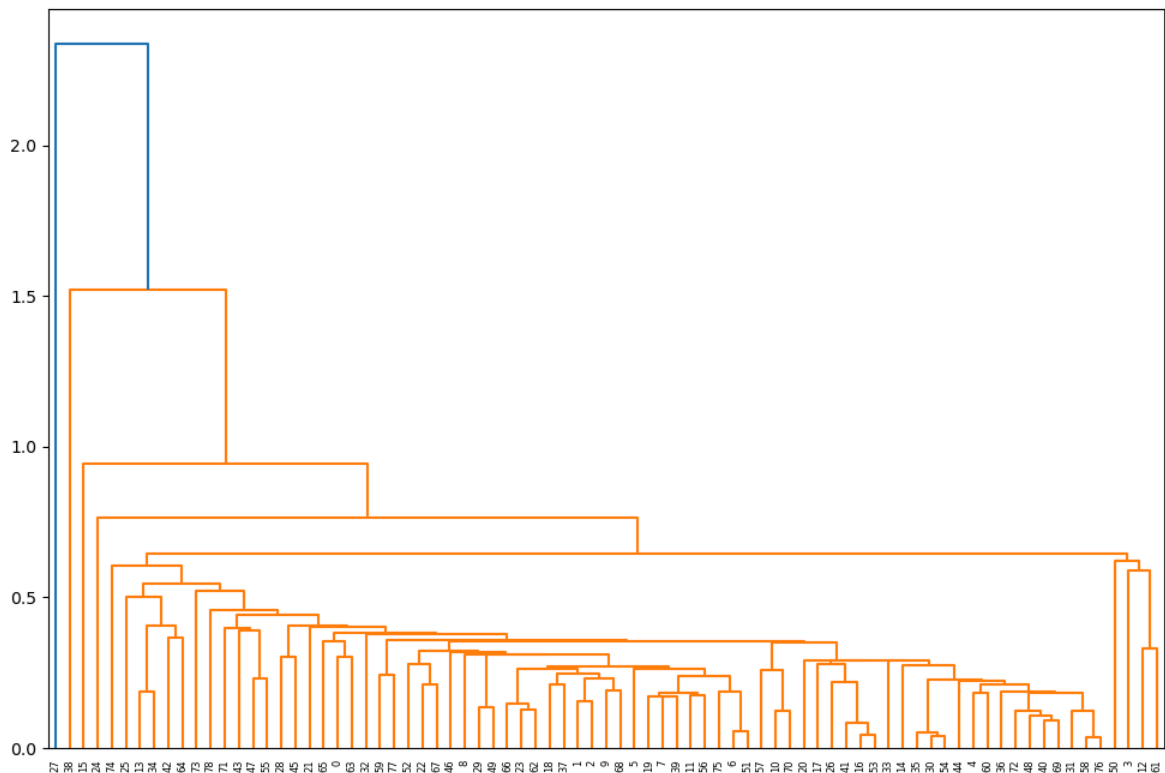
In [34]: `grouped_odi.head()`

Out [34]:

	Player	Ave	SR	Cluster_id
0	SR Tendulkar (INDIA)	44.83	86.23	0
1	KC Sangakkara (Asia/ICC/SL)	41.98	78.86	0
2	RT Ponting (AUS/ICC)	42.03	80.39	0
3	ST Jayasuriya (Asia/SL)	32.36	91.20	2
4	DPMD Jayawardene (Asia/SL)	33.37	78.96	1

In [35]:

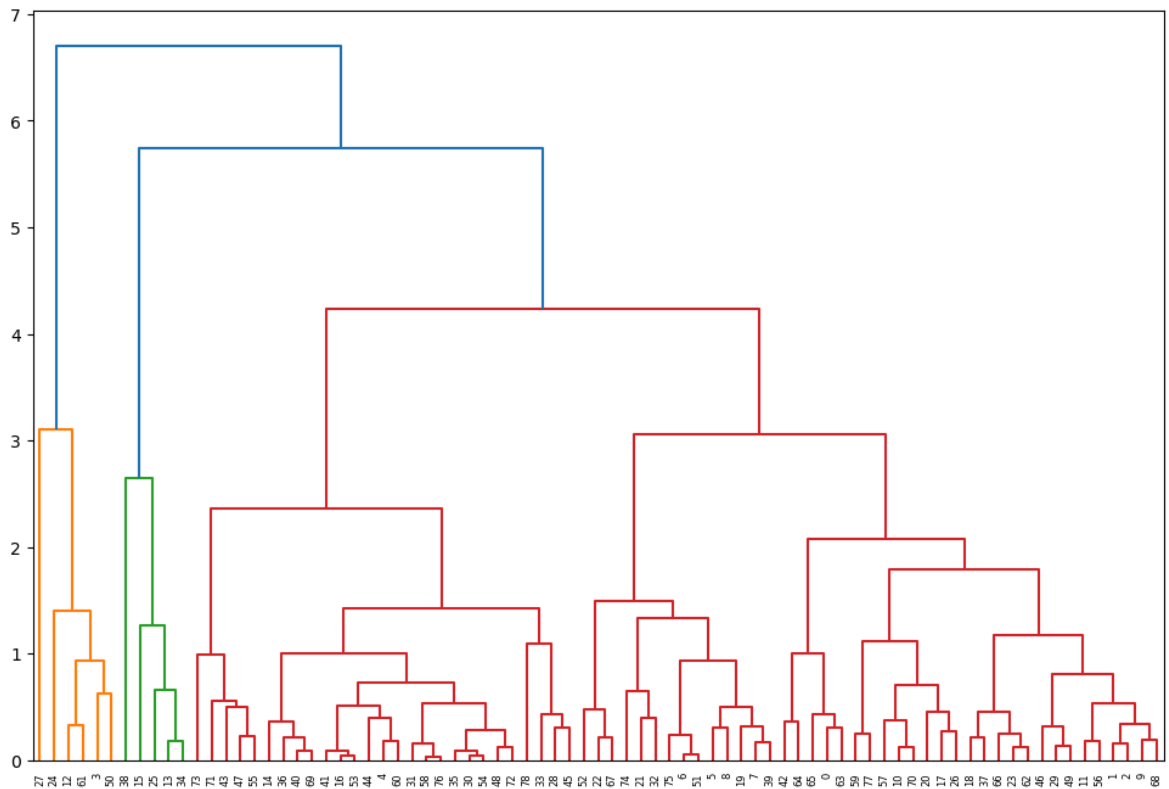
```
# single Linkage
plt.figure(figsize = [12,8])
mergings = linkage(odi_scaled, method = 'single', metric = 'euclidean')
dendrogram(mergings)
plt.show()
```



In [36]:

```
# Complete linkage
plt.figure(figsize = [12,8])
mergings = linkage(odi_scaled, method = 'complete', metric = 'euclidean')
```

```
dendrogram(mergings)
plt.show()
```



```
In [37]: cut_tree(mergings, n_clusters = 3).shape
```

```
Out[37]: (79, 1)
```

```
In [38]: cluster_labels = cut_tree(mergings, n_clusters =3).reshape(-1,)
cluster_labels
```

```
Out[38]: array([0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 2, 0, 2, 0, 0, 0, 0, 0, 0,
                0, 0, 1, 2, 0, 1, 0, 0, 0, 0, 0, 0, 0, 2, 0, 0, 0, 2, 0, 0, 0, 0, 0,
                0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0,
                0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```

```
In [39]: grouped_odi['Cluster_labels'] = cluster_labels
grouped_odi.head()
```

```
Out[39]:
```

	Player	Ave	SR	Cluster_id	Cluster_labels
0	SR Tendulkar (INDIA)	44.83	86.23	0	0
1	KC Sangakkara (Asia/ICC/SL)	41.98	78.86	0	0
2	RT Ponting (AUS/ICC)	42.03	80.39	0	0
3	ST Jayasuriya (Asia/SL)	32.36	91.20	2	1
4	DPMD Jayawardene (Asia/SL)	33.37	78.96	1	0

```
In [40]: fig = px.scatter(grouped_odi, x = 'Ave', y = 'SR', trendline = 'ols', hov
                title = "ODI Clusters", width = 1000, height = 700)
fig.show()
```



```
In [41]: grouped_odi[grouped_odi['Cluster_labels'] == 0]
```

Out [41]:

	Player	Ave	SR	Cluster_id	Cluster_labels
0	SR Tendulkar (INDIA)	44.83	86.23	0	0
1	KC Sangakkara (Asia/ICC/SL)	41.98	78.86	0	0
2	RT Ponting (AUS/ICC)	42.03	80.39	0	0
4	DPMD Jayawardene (Asia/SL)	33.37	78.96	1	0
5	Inzamam-ul-Haq (Asia/PAK)	39.52	74.24	1	0
...	...	...	...	...	...
74	CG Greenidge (WI)	45.03	64.92	0	0
75	Misbah-ul-Haq (PAK)	43.40	73.75	0	0
76	PD Collingwood (ENG)	35.36	76.98	1	0
77	A Symonds (AUS)	39.75	92.44	2	0
78	Abdul Razzaq (Asia/PAK)	29.70	81.25	1	0

68 rows x 5 columns

In [42]: `grouped_odi[grouped_odi['Cluster_labels'] == 1]`

Out [42]:

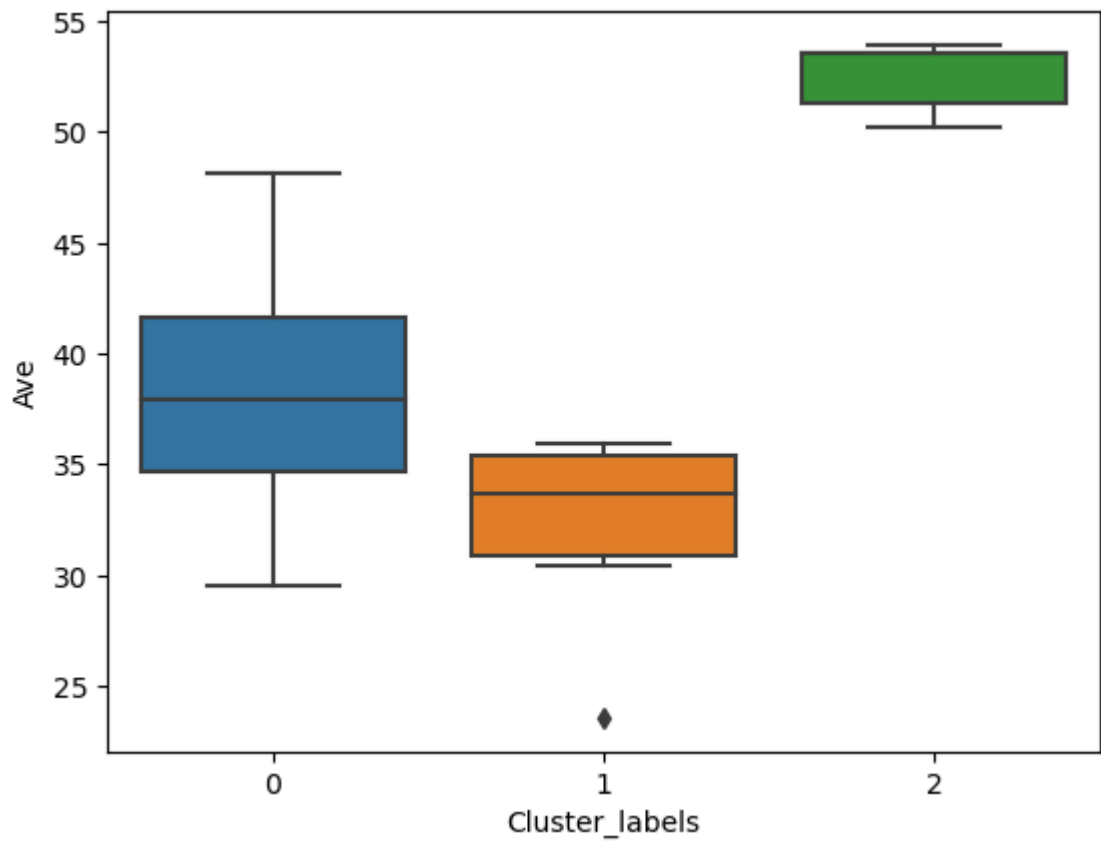
	Player	Ave	SR	Cluster_id	Cluster_labels
3	ST Jayasuriya (Asia/SL)	32.36	91.20	2	1
12	AC Gilchrist (AUS/ICC)	35.89	96.94	2	1
24	V Sehwag (Asia/ICC/INDIA)	35.05	104.33	2	1
27	Shahid Afridi (Asia/ICC/PAK)	23.57	117.00	2	1
50	BB McCullum (NZ)	30.41	96.37	2	1
61	SK Raina (INDIA)	35.46	93.76	2	1

In [43]: `grouped_odi[grouped_odi['Cluster_labels'] == 2]`

Out [43]:

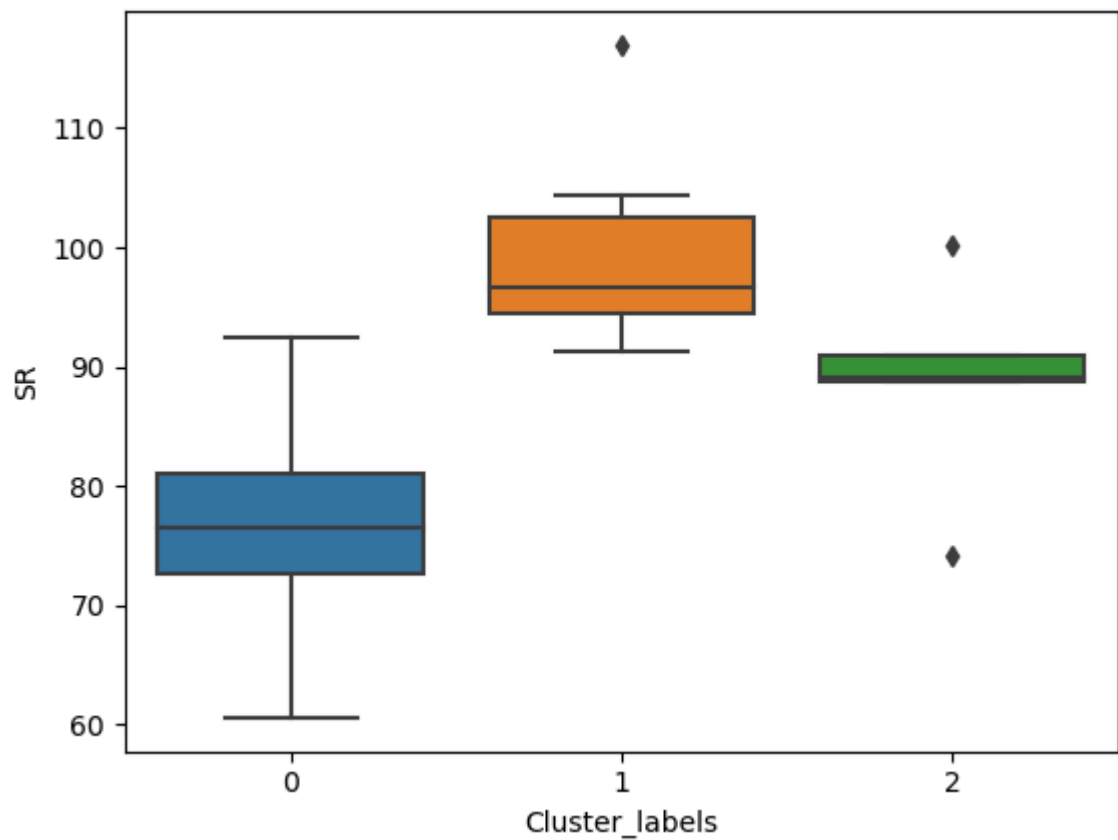
	Player	Ave	SR	Cluster_id	Cluster_labels
13	MS Dhoni (Asia/INDIA)	51.32	88.69	0	2
15	AB de Villiers (Afr/SA)	53.55	100.25	0	2
25	V Kohli (INDIA)	53.94	90.99	0	2
34	HM Amla (SA)	50.25	89.05	0	2
38	MG Bevan (AUS)	53.58	74.16	0	2

In [44]: `sns.boxplot(x='Cluster_labels', y = 'Ave', data = grouped_odi)`Out [44]: `<Axes: xlabel='Cluster_labels', ylabel='Ave'>`



```
In [45]: sns.boxplot(x='Cluster_labels',y = 'SR', data = grouped_odi)
```

```
Out[45]: <Axes: xlabel='Cluster_labels', ylabel='SR'>
```



```
In [50]: # KNN Algorithm
```

```

from sklearn.neighbors import NearestNeighbors
from random import sample
from numpy.random import uniform
import numpy as np
from math import isnan # is not a number

```

The `hopkins` function calculates the Hopkins statistic to assess the degree of clustering tendency in a dataset X . It measures whether the dataset is uniformly distributed (random) or has a cluster structure.

This `hopkins` function calculates the **Hopkins statistic** to assess the degree of clustering tendency in a dataset (X). It measures whether the dataset is uniformly distributed (random) or has a cluster structure.

1. Initialize Parameters:

- `d` is the number of features in (X).
- `n` is the total number of data points in (X).
- `m` is set as 10% of (n), the number of sample points used to calculate the statistic.

2. Compute Nearest Neighbors:

- `nbrs = NearestNeighbors(n_neighbors=1).fit(X.values)` trains a nearest neighbor model on (X).

3. Random Samples:

- `rand_X = sample(range(0, n, 1), m)` randomly selects (m) points from (X) for comparison.

4. Distances for Random and Actual Points:

- `ujd` (random points) and `wjd` (sample points from (X)) store distances for calculating the Hopkins statistic.
- For each randomly generated point and each actual data point:
 - A point `ujd` is generated randomly within the feature range, and its nearest neighbor distance from `X` is calculated.
 - `wjd` stores the nearest neighbor distance for a random sample from (X) itself.

5. Calculate Hopkins Statistic (H):

- ($H = \frac{\sum ujd}{\sum ujd + \sum wjd}$)
- If ($H \approx 0.5$), the data is likely randomly distributed.
- If (H) is close to 1, the data has a strong clustering structure.

6. Handle NaNs:

- If (H) is NaN, it resets (H) to 0, which may indicate an issue with distance calculations.

The Hopkins statistic is commonly used to test if a dataset is appropriate for clustering (with values near 1 indicating a higher likelihood of cluster presence).

```
In [66]: def hopkins(X):
    d = X.shape[1]
    n = len(X)
    m = int(0.1 * n)
    nbrs = NearestNeighbors(n_neighbors = 1).fit(X.values)

    rand_X = sample(range(0,n,1),m)

    ujd = []
    wjd = []
    for j in range(0,m):
        u_dist, _ = nbrs.kneighbors(uniform(np.amin(X,axis =0), np.amax(X,axis =0), (d,1)))
        ujd.append(u_dist[0][1])
        w_dist, _ = nbrs.kneighbors(X.iloc[rand_X[j]].values.reshape(1,-1))
        wjd.append(w_dist[0][1])
    H = (sum(ujd) / (sum(ujd)+sum(wjd)))
    if isnan(H):
        print(ujd, wjd)
        H = 0
    return H
```

```
In [68]: hopkins(odi_scaled)
```

```
Out[68]: 0.8001953059750758
```

A Hopkins value of approximately 0.8 indicates a strong clustering tendency in dataset odi_scaled.

Hopkins Statistic Range: The Hopkins statistic generally ranges from 0 to 1.

Values close to 0.5 suggest a random distribution (no clustering). Values closer to 1 (typically >0.75 indicate a significant cluster structure in the data. Values closer to 0 indicate data that's regularly spaced (anti-clustered). Since obtained value is 0.8 it suggests that the dataset odi_scaled is likely well-suited for clustering algorithms, as it contains natural clusters.

```
In [ ]:
```